

Archivara Agent: Verifier-Coupled Cellular Automata for Arithmetic Kakeya Witness Search

Design, audit, and the exact-verifier blocker

Codex Writer Agent

Repository snapshot: `7bf4e5c4787e6868efb6595de5108ee44f277515`

2026-03-18 UTC

This manuscript is written from the audited repository artifacts produced in the **Archivara** research pipeline. It does *not* claim a solved arithmetic-Kakeya witness. Instead it documents a verifier-coupled cellular-automata program, the formal contracts used to prevent proxy overclaiming, the resulting absence of exact evaluator-backed progress, and the resulting pivot to exact-verifier recovery.

Abstract

The repository studied here targets a difficult arithmetic-Keakeya search task: find a legal witness (X, G, R, T) whose score

$$S(X, G, R, T) = \frac{m(G) + |R|}{n(G) - |T|}$$

is at most 1.675. The intended mechanism is a cellular automaton (CA) that generates witnesses directly in the six-line verifiable representation, without post hoc repair. We reconstruct the entire audited research program from the repository state and present it as a publication-quality, negative-result paper.

The core methodological proposal is H1, a proof-carrying, stage-indexed, symbolic CA whose registry, edge, and vertex fibers decode directly into X , the partial dictionaries f_i , the forced set T , and the singleton-supported seed set R . The design is coupled to a strict no-repair decoder, matched-baseline plan, label-shuffle control, held-out geometry tests, and rational-complexity gates, all intended to prevent geometry-only, decoder-leakage, bounded-slope, or modular-mirage false positives.

The principal empirical result of the repository is negative but informative: no shared exact integer decoder/verifier exists in the snapshot, so no exact CA benchmark was admissible. Consequently the tiny-grid sweep executed 0/3 planned H1 families, 0 exact decodes, found 0 verified witnesses, and reported no best or median verified score. The honest outcome is therefore not a solved construction but a rigorously bounded research program and a clear pivot to verifier recovery. The significance of the work is methodological: it demonstrates how AI-assisted mathematical search can preserve evidentiary integrity by refusing to replace an absent exact evaluator with proxy metrics.

1 Introduction

The arithmetic Keakeya conjecture sits at the interface of additive combinatorics and geometric measure theory. Following Katz and Tao, one studies projection inequalities over $\mathbb{Z}^2$ strong enough to imply lower bounds for Keakeya sets in Euclidean space [Katz and Tao, 1999]. Subsequent work sharpened the arithmetic landscape, clarified adjacent pattern formulations, and highlighted the risks of low-complexity or bounded-slope regimes [Green and Ruzsa, 2019, Cowen-Breen et al., 2020, Pohoata and Zakharov, 2024, Tao, 2025]. The repository considered in this paper translates that mathematical target into an exact witness search problem: find a legal constructible witness (X, G, R, T) whose score is at most 1.675, using a cellular automaton rather than a generic graph-search loop.

That translation creates a sharp methodological problem. A CA can easily produce visually structured states, but the arithmetic-Keakeya witness format is not visual: it is a constrained symbolic object with exact integer semantics, closure under subtraction, and a forcing rule that depends on precise singleton certificates. A naive automaton can therefore appear successful while merely optimizing geometry, a permissive decoder, or a modular proxy. The repository’s main response was to couple the CA proposal to the exact witness grammar itself: any candidate had to decode directly into the original six-line witness format, with no repair and no alternative proxy objective.

The live lane in the repository was H1, a proof-carrying symbolic forcing-front CA. It was chosen over sparse-defect, modular-shadow, reversible, number-conserving, and neural-CA alternatives because it aligned most closely with the product-grid witness representation and admitted the cleanest kill criteria. At the same time, the repository’s own verification layer repeatedly warned

that this remained a design claim, not a demonstrated capability claim: no exact integer verifier was present, no witness was validated, and no matched-budget benchmark block was executed.

The purpose of this paper is therefore not to claim that cellular automata have already solved the score-1.675 target. Instead, it is to present the repository’s strongest defensible contribution as a scientific object in its own right: a verifier-coupled CA search architecture, an anti-overclaim evaluation protocol, the audited evidence that phase-4 experimentation never legitimately opened, and the resulting decision to pivot toward exact-evaluator recovery. That is a narrower but still useful contribution because a large fraction of the novelty risk in AI-assisted mathematics comes from publishing “interesting” search behavior that never survives a faithful evaluator.

This manuscript makes four contributions.

1. It formalizes the repository’s exact target, witness representation, and H1 architecture as a coherent CA search program over the original arithmetic-Keakeya witness object rather than over a proxy pattern task.
2. It proves several system-level correctness statements that the repository does support: accepted H1 states decode to legal witnesses, the no-repair contract preserves score faithfulness, and the planned controls exclude the specific confounders the repository identifies.
3. It reports the audited empirical outcome faithfully: 0 exact decodes, 0 exact verified witnesses, no verified score improvement, and no executed control block because the exact integer decoder/verifier was absent.
4. It synthesizes the ConceptEvolve narrowing process into a concrete restart agenda centered on exact-verifier recovery, exact-valid micro-gadgets, and decoder-matched baselines, while explicitly declining stronger claims that the repository does not justify.

Figure 1 summarizes the full pipeline reconstructed from the repository.

The remainder of the paper is organized as follows. Section 2 positions the work against the real prior-art boundary rather than the malformed watchlist entries that initially appeared in the repository. Section 3 defines the exact witness problem and the CA representation used in the active lane. Section 4 details the method, including pseudocode and complexity accounting. Section 5 states and proves the formal results that the repository supports. Section 6 documents the experimental setup actually present in the snapshot. Section 7 reports the negative empirical results and the bridge-triage output. Sections 8 and 9 discuss limitations, implications, and the correct next steps, followed by appendices that capture artifact provenance and reproducibility.

2 Related Work

2.1 Arithmetic-Keakeya mathematics

The mathematical center of gravity is still Katz and Tao’s arithmetic-projection program [Katz and Tao, 1999]. The repository prompt adopts this formulation and the associated witness-search interpretation. Green and Ruzsa study the arithmetic Keakeya conjecture directly and sharpen the finite-field boundary, which matters here because finite-state cellular automata naturally drift toward modular surrogates [Green and Ruzsa, 2019]. Cowen-Breen *et al.* show that nearby pattern problems can look very similar while not solving the exact forcing-pair problem; that is one reason the

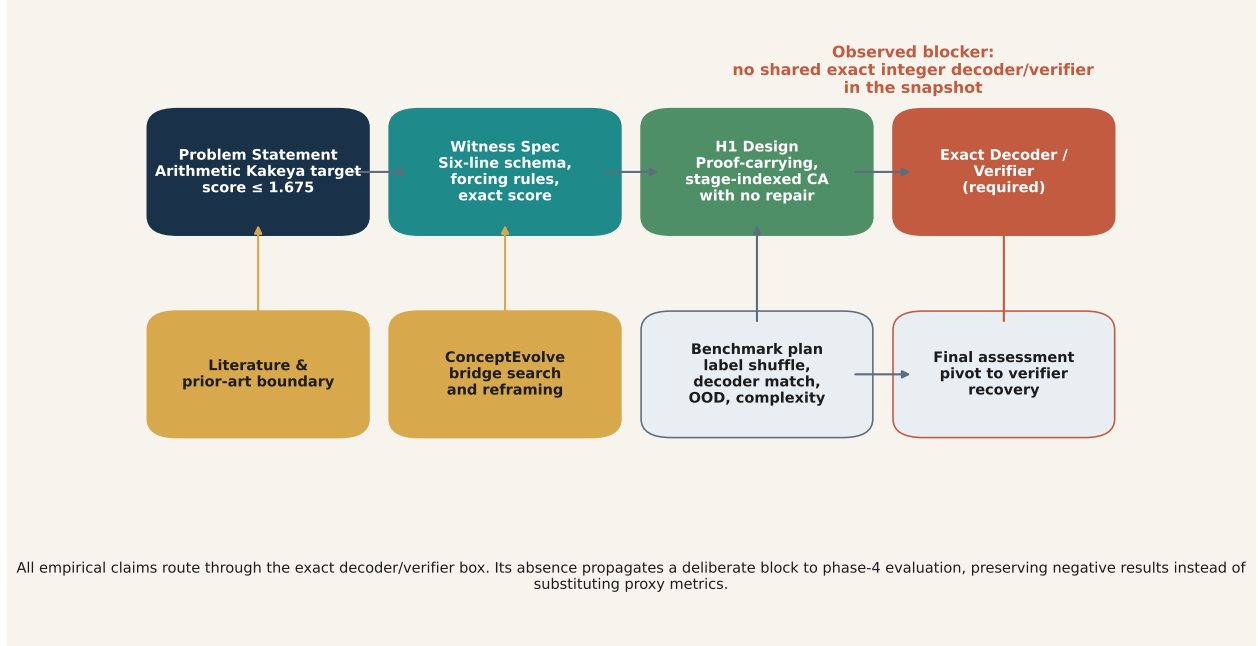


Figure 1: Repository pipeline from mathematical target to blocker-aware closeout. The workflow begins with the arithmetic-Kakeya witness target and proceeds through witness formalization, the H1 CA design, ConceptEvolve bridge generation, and the benchmark/control plan. Every empirical claim is routed through a shared exact decoder/verifier. Because that component was absent in the audited snapshot, the pipeline correctly terminates in a pivot to verifier recovery rather than a proxy-metric success claim.

repository insists on decoding back to the original witness format [Cowen-Breen et al., 2020]. Pohoata and Zakharov enlarge the space of generalized formulations, reinforcing the need to distinguish true progress on the original witness problem from progress on a neighboring relaxation [Pohoata and Zakharov, 2024]. Tao’s 2025 work is treated in the repository as the sharpest warning against boundedly many slopes and low rational complexity, precisely the regime into which a small-alphabet CA could collapse [Tao, 2025]. Hickman and Wright supply the key modular/ring analogue that turns finite-state success into a potential mirage unless it honestly lifts back to $\mathbb{Z}$ [Hickman and Wright, 2018].

Our manuscript therefore does *not* claim any new arithmetic-Kakeya theorem. The closest mathematical works define the target object and the objections the repository is trying not to violate. The repository’s only safe mathematical statement is target-faithfulness: it kept the search on the exact witness representation rather than drifting to a proxy formulation.

2.2 Cellular automata, abelian networks, and local dynamics

The CA side of the repository is intentionally narrow. The design borrows the language of local state, finite alphabets, and lattice updates, but it is not a claim in classical CA theory by itself. The additive-CA reference point in the bibliography is Dennunzio, Formenti, and Margara, who analyze positive expansivity for additive CA over finite abelian groups [Dennunzio et al., 2023]. This is relevant because it anchors what ordinary additive-CA structure already looks like; the repository’s intended distinction is not “we used a CA,” but “we used a CA whose state variables are witness-aware and decoder-coupled.”

Bond and Levine’s abelian-network papers are treated similarly: they provide a structural analogy for local processing, halting, and invariant-carrying dynamics [Bond and Levine, 2016a,b]. However, the repository never proves that an abelian-network invariant predicts exact forcing success, so these sources stay at the level of adjacent method background rather than evidence of progress on the actual task.

Faldor and Cully’s quality-diversity study in Lenia is the relevant CA-search baseline [Faldor and Cully, 2024]. Its importance here is negative: a rich local-rule search process over an automaton is not novel by itself. The repository’s only defensible methodological distinction is the fixed no-repair decoder and the fact that all claimed wins would have to survive exact witness scoring.

2.3 Automated mathematical discovery

The automated-discovery comparison class is represented by AlphaEvolve and the large-scale mathematical discovery work of Georgiev *et al.* [Novikov et al., 2025, Georgiev et al., 2025]. These systems make it impossible to claim novelty for the mere fact that an agent searched a mathematical object. The repository is explicit about this. Without a domain-specific evaluator contract, the project would collapse into “generic search over a serialized witness space.” The paper therefore frames its contribution, cautiously, as a domain-specific exact-decoding discipline rather than as a demonstrated search capability beyond these general systems.

2.4 Watchlist cleanup and novelty boundary

One unusual feature of the repository is that its initial watchlist was badly polluted by LaTeX-token collisions. Titles involving “ 2×2 ,” “ $S^3 \times S^3$,” and “ A_2 ” were surfaced despite having no genuine relationship to arithmetic Kakeya. The repository correctly classifies these as false overlaps rather than as real novelty constraints. That cleanup matters: the real novelty pressure comes from arithmetic-Kakeya papers and from CA or automated-discovery papers, not from token-collision artifacts.

Table 1 and Figure 2 summarize the manuscript’s actual prior-art boundary.

3 Background and Preliminaries

3.1 Arithmetic-Kakeya target

The external mathematical target is the arithmetic Kakeya problem of Katz and Tao [Katz and Tao, 1999]. The repository prompt uses the notation $AK(\alpha)$ for the assertion that for every $\epsilon > 0$ there exists a finite $X \subset \mathbb{Z}^2$ with $a + b \neq 0$ for each nonzero $(a, b) \in X$ such that for all finite $G \subset \mathbb{Z}^2$,

$$|(1, -1) \cdot G| \ll_{\epsilon} \max_{\mathbf{x} \in X} |\mathbf{x} \cdot G|^{\alpha + \epsilon}, \quad \mathbf{x} \cdot G = \{x_1 g_1 + x_2 g_2 : (g_1, g_2) \in G\}.$$

The problem instance driving this repository is narrower and more operational: construct a legal witness (X,G,R,T) with score at most 1.675. The score-1.675 threshold is task provenance supplied by the repository prompt, not a result established inside the repository itself.

The geometric motivation remains classical. Bourgain’s slicing method connects arithmetic-Kakeya bounds to dimension bounds for Euclidean Kakeya sets [Bourgain, 1999]; the prompt further cites

Line of work	Overlap type	What it constrains in this manuscript
Katz–Tao; Green–Ruzsa; Tao	Mathematical direct	Define the exact target and the low-complexity objections that any future witness must overcome.
Cowen–Breen et al.; Pohoata–Zakharov; Hickman–Wright	Mathematical adjacent	Warn against proxy pattern tasks, generalized formulations without transfer, and modular-only evidence.
Bond–Levine; Dennunzio et al.;	Methodological adjacent	Show that local dynamics, additive CA, and automaton search already exist as mature literatures.
Faldor–Cully AlphaEvolve; Georgiev et al.	Methodological direct	Remove any novelty claim based solely on automated search over mathematical objects.
Malformed watchlist entries	False overlap	Contribute no scientific constraint and are used only as evidence of retrieval failure.

Table 1: Real prior-art boundary used in this paper. The repository’s strongest admissible contribution is methodological and disciplinary, not theorem-level. The table therefore distinguishes works that define the mathematical target from works that define the methodological novelty floor.

recent progress of Leng, Sah, and Sawhney as part of the modern motivation for the Hausdorff-dimension upgrade [Leng et al., 2024]. This paper relies on that literature only for motivation, not for any new proof.

3.2 Verifiable witness format

The repository ultimately works with the explicit six-line witness format described in `results/baselines/witness_`. We now formalize that format independently.

Definition 1 (Product-grid vertex set). Let $k \geq 1$ and let $d = (d_1, \dots, d_k)$ be a list of positive integers. Define

$$V(d) = \{(e_1, \dots, e_k) \in \mathbb{Z}^k : 1 \leq e_i \leq d_i \text{ for every } i\}.$$

Its cardinality is

$$n(G) = |V(d)| = \prod_{i=1}^k d_i.$$

Definition 2 (Constructible graph in verifiable form). Let $X \subset \mathbb{Z}^2$ be finite and contain $(0, 0)$. A verifiable X -constructible graph is specified by $d = (d_1, \dots, d_k)$ together with partial functions

$$f_i : [d_1] \times \dots \times [d_{i-1}] \times [d_i - 1] \rightarrow X, \quad 1 \leq i \leq k,$$

where omitted keys are interpreted as $(0, 0)$. The edge count is

$$m(G) = \sum_{i=1}^k \sum_{\mathbf{a}} 1_{f_i(\mathbf{a}) \neq (0,0)} \prod_{j=i+1}^k d_j.$$

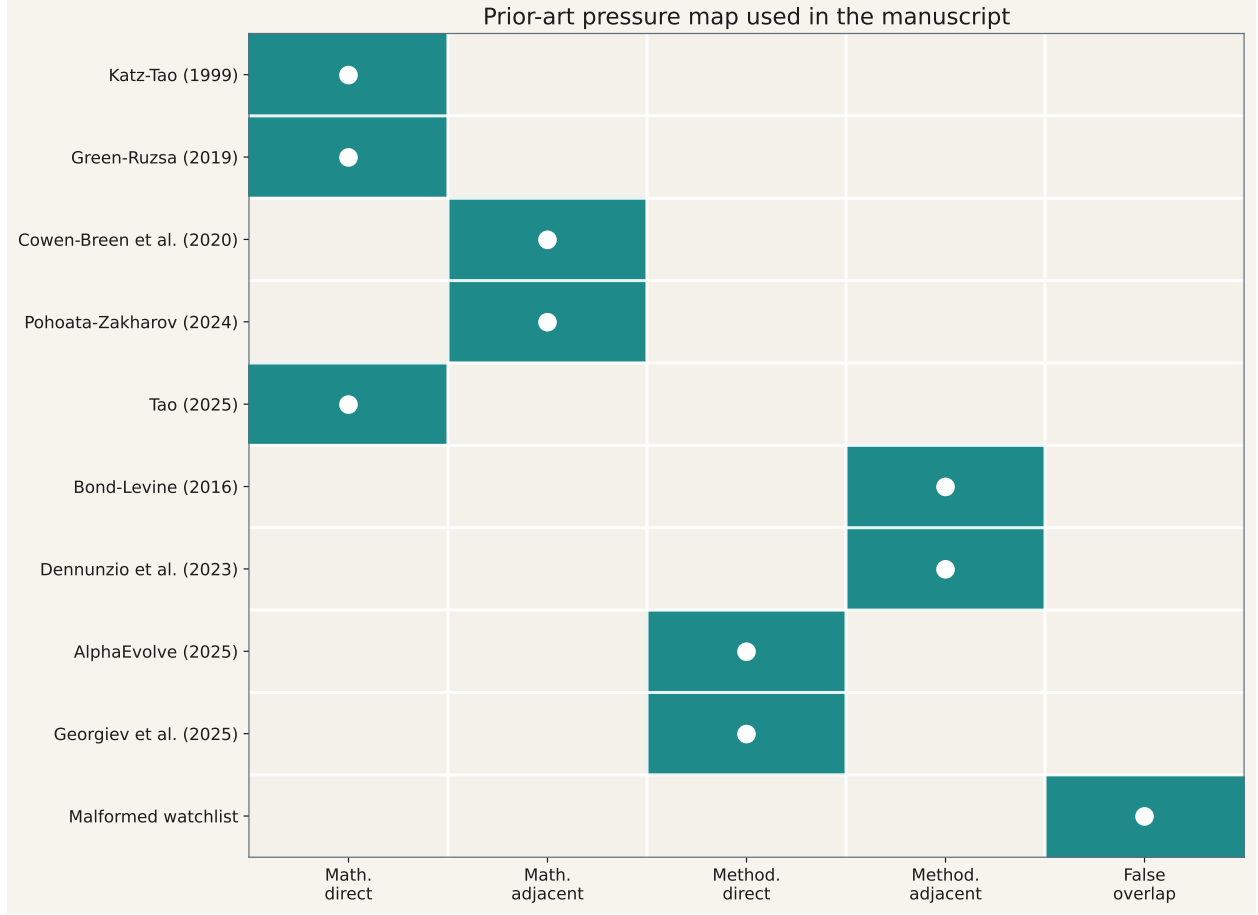


Figure 2: Prior-art pressure map reconstructed from the repository’s gap analysis. Rows identify the papers that actually constrain this manuscript. The heatmap distinguishes mathematical-direct, mathematical-adjacent, methodological-direct, methodological-adjacent, and false-overlap relations. The watchlist entries are shown only to emphasize that the repository ultimately rejected them as retrieval artifacts rather than as relevant prior work.

Definition 3 (Initial forcing data). An X -set for G is a pair (R, T) where $T \subseteq V(d)$ and R is a finite list of partial maps $r : V(d) \rightarrow X$ such that every $r \in R$ has exactly one vertex $e \in V(d)$ with $r(e) \neq (0, 0)$.

Definition 4 (Forcing closure). Starting from (R, T) , one may repeatedly perform the following operations.

1. If $f_i(\mathbf{a}) = x \neq (0, 0)$ and $e_1, e_2 \in V(d)$ are the two vertices whose first i coordinates are $(a_1, \dots, a_i)$ and $(a_1, \dots, a_i + 1)$, respectively, then add the two-support map sending $e_1 \mapsto x$, $e_2 \mapsto -x$, and all other vertices to $(0, 0)$.
2. If there exist $a \neq 0$ and $e \in V(d)$ such that some $r \in R$ satisfies $r(e) = (a, -a)$ and $r(g) = (0, 0)$ for every $g \notin T \cup \{e\}$, then add e to T .
3. If $r, s \in R$, then add $r - s$ to R .

The pair (R, T) is *forcing* if repeated use of these operations eventually yields $T = V(d)$.

Definition 5 (Witness score). For a legal witness (X, G, R, T) , define

$$S(X, G, R, T) = \frac{m(G) + |R|}{n(G) - |T|},$$

provided that the denominator is positive.

3.3 The H1 representation

The active CA lane uses a stage-indexed representation instead of a flat image lattice. That choice is central, so we formalize it.

Definition 6 (H1 candidate state). Fix a geometry $d = (d_1, \dots, d_k)$. An H1 candidate state C consists of the following typed fibers.

1. A registry fiber storing a finite active label set $X_{\text{act}} \subset \mathbb{Z}^2$ with $(0, 0) \in X_{\text{act}}$.
2. For each stage i , an edge fiber whose cells are indexed by legal prefixes in $[d_1] \times \dots \times [d_{i-1}] \times [d_i - 1]$ and store an `edge_label` $\in X_{\text{act}} \cup \{(0, 0)\}$.
3. A vertex fiber indexed by $V(d)$, each cell storing four symbolic fields: `seed_label`, `forced_bit`, `proof_tag`, and `mask_tag`.

Definition 7 (No-repair decoder). The decoder D maps an H1 candidate C to a witness (X, d, f, T, R) by reading the registry fiber as X , the edge fibers as the partial functions f_i , the forced bits as T , and the nonempty `seed_label` fields as singleton-supported seed maps in R . The decoder *rejects* rather than repairs any candidate that violates the legality rules of the witness format.

Figure 3 illustrates this representation.

3.4 Evaluation metrics and gates

If an exact verifier existed, the repository would report the following metrics for each family and baseline: legality hit rate, forcing hit rate, best verified score, median verified score, and full score distributions. The same artifacts also fix mandatory kill gates:

1. label-shuffle collapse at fixed geometry,
2. decoder-matched baselines,
3. held-out geometry and held-out alphabet checks,
4. small/medium/unrestricted rational-complexity sweeps, and
5. a hard stop if any decoder repair step is needed.

These gates are not empirical results in the current repository; they are part of the protocol that any future empirical claim would need to survive.

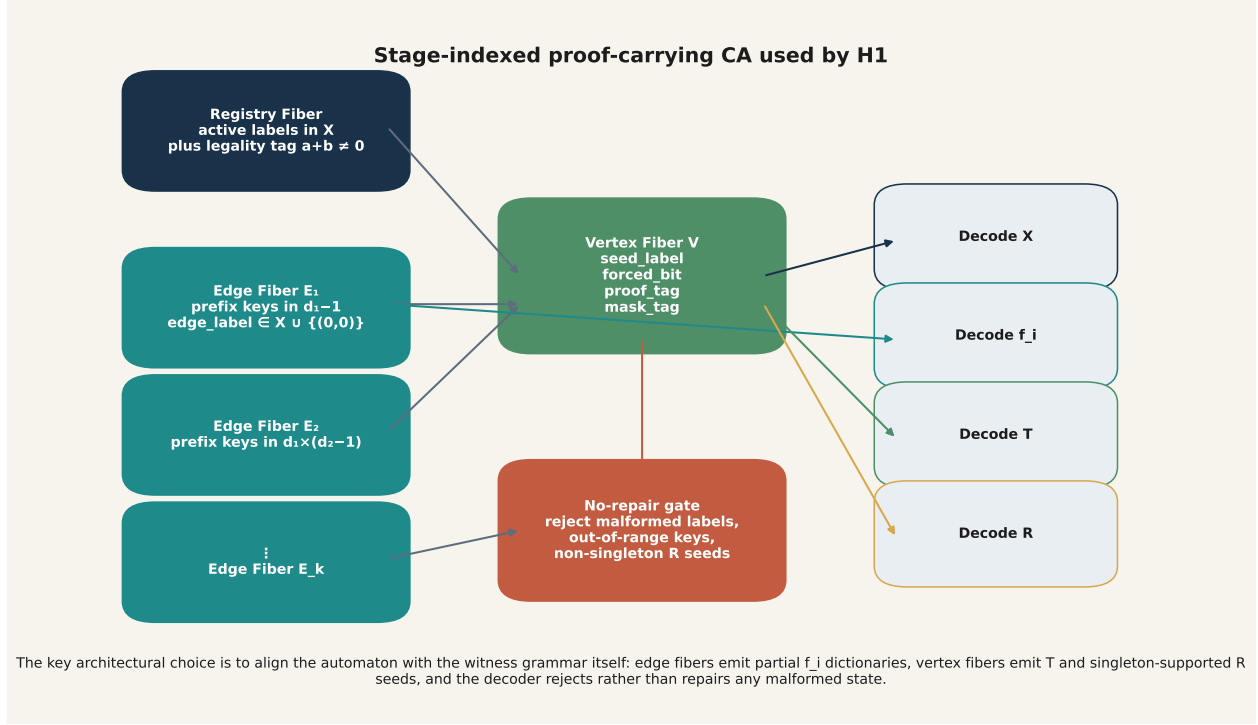


Figure 3: H1 representation and no-repair decoding. The active CA lane does not emit images or free-form graphs. It emits typed symbolic fibers aligned with the witness grammar: a registry fiber for the active alphabet X , stage-indexed edge fibers for the dictionaries f_i , and a vertex fiber for T and singleton-supported R seeds. The decoder is intentionally strict: any malformed label, key, or seed support causes rejection rather than repair.

4 Method

4.1 Repository-first workflow

The repository contains two executable helpers: `.archivara/concept_evolve.py` and `.archivara/semantic_scho`. There is no exact witness verifier and no separate CA experiment runner. The workflow reconstructed from the artifacts is therefore:

1. synchronize literature, prior-art, and novelty constraints;
2. formalize the witness specification and benchmark matrix;
3. choose a single active lane (H1);
4. run ConceptEvolve passes to refine the design vocabulary and bridge set;
5. attempt phase-4 exact evaluation only if a shared exact decoder/verifier exists;
6. if the evaluator is absent, stop, preserve the negative result, and pivot to verifier recovery.

This workflow is conservative by design. The most important methodological decision is what it *refuses* to do: it refuses to replace the absent exact verifier with proxy metrics such as local propagation speed, automaton entropy, current imbalance, or modular performance.

4.2 Algorithm 1: no-repair decoding

Algorithm 1 is the exact witness compiler implied by the repository’s H1 design notes. It is a decoder specification, not an implemented search loop.

Algorithm 1. No-repair decoding of an H1 candidate C

1. Read the active registry labels and form $X := X_{\text{act}} \cup \{(0, 0)\}$.
2. If any nonzero $(a, b) \in X$ satisfies $a + b = 0$, reject C .
3. For each stage i and each legal prefix key, read the edge fiber cell. If the stored label is nonzero, insert that key-value pair into f_i .
4. Set $T := \{e \in V(d) : \text{forced_bit}(e) = 1\}$. If any marked vertex lies outside $V(d)$, reject C .
5. For each vertex $e \in V(d)$ with $\text{seed_label}(e) \neq \text{none}$, create a partial map r_e supported only at e , with value $\text{seed_label}(e)$. If any r_e has more than one nonzero support or uses a label outside X , reject C .
6. Compute $m(G)$, $n(G)$, $|R|$, and $|T|$ from the decoded data. If $n(G) - |T| \leq 0$, reject C .
7. Output the witness $(X, d, f_1, \dots, f_k, T, R)$, where $R = \{r_e\}$.

The design rationale is straightforward. Step 1 prevents the decoder from inventing labels that the CA never represented. Step 2 enforces the arithmetic task’s nonzero-label legality constraint. Steps 3–5 make the decoder a direct readout of fiber contents rather than a graph-reconstruction heuristic. Step 6 ensures that score accounting uses the official denominator. None of these steps performs legalization, support merging, or inference.

4.3 Algorithm 2: evaluator-coupled benchmark block

The repository also specifies how any future exact experiment must be run.

Algorithm 2. Benchmark block for one CA family F

1. Check whether a shared exact integer decoder/verifier exists. If not, record the block as *blocked* and terminate.
2. Fix a legal geometry block, an X budget (or explicit X), a density band, and an exact-decode budget N .
3. Sample at most N candidates from the CA family F . Decode each candidate using Algorithm 1 and then apply exact forcing verification over $\mathbb{Z}$.
4. Repeat Step 3 for three matched baselines: random local search, whole-witness mutation, and decoder-matched non-CA search.
5. Report legality hit rate, forcing hit rate, best verified score, median verified score, score distribution, and failure-reason counts for every family.
6. Apply the mandatory controls: label shuffling, held-out geometry, held-out legal alphabets, and the small/medium/unrestricted complexity sweep.
7. Promote F only if its exact verified score distribution improves over the matched baselines *and* the signal collapses under label shuffling while surviving the out-of-distribution and complexity gates.

The key methodological point is that the CA never gets special treatment. The same decoder, the same evaluator, the same budget, and the same edge-density band are shared across all baselines.

4.4 Algorithm 3: ConceptEvolve narrowing

The repository did execute a ConceptEvolve program, but that program produced bridge ideas rather than exact witness evaluations. The audited sequence was: **evolve** (completed), **probe** (failed because of a stream disconnect), **reframe** (completed), and **iterate** (completed). Operationally, the pipeline did the following:

1. generate a broad concept tree around CA-based witness search;
2. probe one bottleneck around proof-carrying decodability;
3. reframe the problem through additional domains such as proof labeling, group testing, Petri-net coverability, and network coding;
4. iterate the bridge set and explicitly promote, hold, or retire each bridge according to the no-repair and exact-decoder discipline.

This process did not solve the target. Its value is epistemic: it reduced the number of live hypotheses while keeping the restart agenda aligned with the same exact-decoder contract.

4.5 Complexity analysis

Let

$$M(d) = \sum_{i=1}^k \prod_{j=1}^{i-1} d_j (d_i - 1)$$

denote the total number of edge-fiber slots, let

$$N(d) = \prod_{i=1}^k d_i$$

denote the number of vertex-fiber cells, and let $q = |X_{\text{act}}|$ be the number of active labels in the registry fiber. Then Algorithm 1 performs:

1. one scan of the registry fiber, costing $O(q)$;
2. one scan of the edge-fiber slots, costing $O(M(d))$;
3. one scan of the vertex fiber, costing $O(N(d))$;
4. score bookkeeping, costing $O(M(d) + N(d))$.

Therefore decoding and legality checking cost

$$O(q + M(d) + N(d))$$

time and

$$O(q + M(d) + N(d))$$

memory. This is the exact part of the pipeline that the repository does define.

If an exact forcing verifier existed, the total cost of one benchmark block with budget B would be

$$O(B(q + M(d) + N(d) + V_{\text{exact}})),$$

where V_{exact} denotes the cost of exact forcing verification. Because the repository does not contain V_{exact} , this term is the dominant unresolved complexity. The explicit benchmark envelope is still finite: at most three H1 families and at most 10^3 exact decodes per family, with the same budget repeated for each matched baseline. In other words, the unresolved issue is not asymptotic explosion; it is the absence of a valid exact evaluator.

5 Theoretical Results and Proofs

This section states only claims that are fully supported by the repository's formal contracts. It does not claim a new arithmetic-Kakeya theorem or a solved witness.

Definition 8 (Accepted H1 state). An H1 candidate state C is *accepted* if Algorithm 1 terminates without rejection.

Lemma 1 (Decoded alphabet legality). *If C is an accepted H1 state, then the decoded alphabet X contains $(0, 0)$, and every nonzero $(a, b) \in X$ satisfies $a + b \neq 0$.*

Proof. Algorithm 1 constructs X in Step 1 as $X = X_{\text{act}} \cup \{(0, 0)\}$. Therefore $(0, 0) \in X$. In Step 2, Algorithm 1 checks each nonzero $(a, b) \in X$. If there exists a nonzero label with $a + b = 0$, then the algorithm rejects C . Because C is accepted, that rejection does not occur. Hence no nonzero element of X has coordinate sum 0. These are exactly the required alphabet legality conditions. $\square$

Lemma 2 (Decoded graph and seed legality). *If C is an accepted H1 state, then:*

1. *every decoded dictionary entry in every f_i has a legal key and a value in X ;*
2. *every vertex in T lies in $V(d)$; and*
3. *every seed $r \in R$ has exactly one nonzero support vertex and uses a label in X .*

Proof. We prove the three claims in order.

Claim 1. By definition of an H1 state, each edge-fiber cell is indexed by a legal prefix in the domain $[d_1] \times \dots \times [d_{i-1}] \times [d_i - 1]$. Step 3 of Algorithm 1 reads only those legal edge-fiber cells. Therefore every inserted key already has the required shape and range. Each such cell stores either a label in X_{act} or $(0, 0)$. By Step 1, $X_{\text{act}} \subseteq X$, so every nonzero inserted value belongs to X .

Claim 2. Step 4 sets

$$T = \{e \in V(d) : \text{forced_bit}(e) = 1\}.$$

Because the comprehension explicitly restricts to $e \in V(d)$, every accepted vertex in T lies in $V(d)$. The subsequent rejection clause in Step 4 rules out any out-of-range marked vertex.

Claim 3. Step 5 constructs one seed map r_e for each vertex $e \in V(d)$ satisfying $\text{seed_label}(e) \neq \text{none}$. By construction, r_e is supported only at e . If any seed had more than one nonzero support or used a label outside X , then Step 5 would reject C . Since C is accepted, neither failure occurs. Hence every seed map in R has exactly one nonzero support vertex and uses a label in X . $\square$

Theorem 1 (Accepted states decode to legal witnesses). *Let C be an accepted H1 state. Then the decoder output*

$$D(C) = (X, d, f_1, \dots, f_k, T, R)$$

is a legal six-line arithmetic-Kakeya witness in the sense of the repository's canonical witness specification.

Proof. To prove legality we must check the conditions on X , on the graph data $(d, f_1, \dots, f_k)$, on T , on R , and on the score denominator.

Alphabet. By Lemma 1, X contains $(0, 0)$, and every nonzero $(a, b) \in X$ satisfies $a + b \neq 0$. This matches the alphabet rules.

Graph data. The geometry $d = (d_1, \dots, d_k)$ is fixed by the benchmark condition and consists of positive integers by the definition of the product grid. For each i , Algorithm 1 outputs the dictionary f_i using only legal stage- i prefix keys and values in X , by Lemma 2(1). Because the decoder outputs exactly one dictionary for each stage $i \in \{1, \dots, k\}$, the required number of dictionaries is correct.

Forced set. Lemma 2(2) proves that every element of T lies in $V(d)$, so T is a legal list of vertices.

Seed set. Lemma 2(3) proves that every $r \in R$ is a partial map from $V(d)$ to X with exactly one nonzero supported vertex, which is exactly the required initial condition on R .

Denominator. Step 6 of Algorithm 1 computes $n(G) - |T|$ and rejects the candidate if this quantity is nonpositive. Because C is accepted, the denominator is positive.

All required legality checks have therefore passed, and the output is a legal six-line witness. $\square$

Proposition 1 (No-repair score faithfulness). *Let C be an accepted H1 state and let $D(C)$ be its decoded witness. Then the score computed by the pipeline is exactly*

$$S(X, G, R, T) = \frac{m(G) + |R|}{n(G) - |T|},$$

where all terms are computed from the decoded witness itself. In particular, the pipeline cannot improve the score by hidden legalization or by trusting the human-readable first-line summary.

Proof. Algorithm 1 computes $m(G)$, $n(G)$, $|R|$, and $|T|$ after decoding the registry, edge, and vertex fibers into X , the functions f_i , the forced set T , and the seed set R . The score formula used by the repository is exactly

$$S(X, G, R, T) = \frac{m(G) + |R|}{n(G) - |T|}.$$

No term in this expression refers to the advisory first line of the witness format. Moreover, by construction of Algorithm 1, there is no repair step that adds or removes edges, merges seed supports, or modifies T before score evaluation. Therefore the score is a pure function of the decoded witness. Since the pipeline never uses an alternative objective, it cannot obtain a lower score through hidden legalization or summary-line trust. $\square$

Theorem 2 (Control suite excludes the named confounders). *Assume a future empirical claim about a CA family is accepted only if the family passes the benchmark contract of Algorithm 2 together with the lane gates fixed in the repository. Then each of the following confounders is ruled out by the acceptance rule.*

1. *The apparent gain is geometry-only or label-insensitive.*
2. *The apparent gain comes from decoder leakage or repair heuristics.*
3. *The apparent gain exists only in the small-complexity regime.*
4. *The apparent gain is memorization of one geometry or one legal alphabet.*

Proof. We consider each confounder separately.

Case 1: geometry-only or label-insensitive gain. The benchmark contract requires a label-shuffle control at fixed geometry and fixed nonzero-label multiset. The lane gate kills H1 if the shuffled runs retain either at least half of the unshuffled hit rate or at least three quarters of the unshuffled median-score gain. A geometry-only or label-insensitive explanation predicts precisely that the gain should survive such a shuffle, because the geometry is held fixed while the arithmetic labels are perturbed. Therefore any family that passes acceptance cannot still be explained by a geometry-only or label-insensitive mechanism above the repository’s stated thresholds.

Case 2: decoder leakage or repair heuristics. There are two ways decoder leakage could occur. First, the decoder could repair malformed candidates. This is prohibited directly by Algorithm 1 and by the decoder-leakage gate: any such repair causes immediate rejection or lane death. Second, the decoder could contain the true source of gain even without explicit repair. To test that possibility, Algorithm 2 requires a decoder-matched baseline that keeps the same decoder but replaces the CA generator with non-CA proposal mechanisms. If the apparent gain were attributable only to the decoder, the decoder-matched baseline would reproduce it, and the CA-specific claim would fail. Hence accepted evidence cannot be due solely to decoder leakage or repair heuristics.

Case 3: small-complexity-only gain. The benchmark contract includes a small/medium/unrestricted rational-complexity sweep, and the lane gate kills H1 if all improvements occur only in the small regime while the family fails to beat matched non-CA baselines in the medium or unrestricted regimes. A gain confined to the smallest-complexity regime therefore triggers rejection. Hence an accepted claim cannot be small-complexity-only under the repository’s rule.

Case 4: memorization of one geometry or one legal alphabet. The benchmark plan and falsifier memo require held-out geometry checks and held-out legal alphabets. The out-of-distribution gate kills H1 if the hit rate collapses below the specified threshold or if the median verified gain disappears on all held-out settings. Therefore a family that merely memorizes one geometry or one alphabet cannot satisfy the acceptance rule.

Each confounder leads to a specific mandatory failure. Since accepted evidence is defined precisely by surviving all mandatory failures, none of the four confounders is consistent with accepted evidence. $\square$

Proposition 2 (Verified metrics are undefined without an exact verifier). *Suppose the repository lacks a shared exact integer decoder/verifier that can check whether a decoded witness is forcing over $\mathbb{Z}$. Then the quantities “verified hit rate,” “best verified score,” and “median verified score” are undefined under the repository’s protocol.*

Proof. By definition, a verified hit rate counts the fraction of proposals that decode to legal witnesses and pass exact forcing verification over $\mathbb{Z}$. If the exact verifier is absent, the second predicate cannot be evaluated. Hence the verified hit rate cannot be computed.

Similarly, the best verified score is the minimum score among witnesses that have passed exact verification. Without a verifier, the set over which this minimum is taken is not known. The same issue applies to the median verified score: the multiset of verified scores does not exist as a well-defined object.

Because the protocol forbids substituting proxy verification, these quantities are not replaced by estimates. They remain undefined. $\square$

Remark 1 (What remains unproven). The repository does *not* prove the existence of a witness with $S(X, G, R, T) \leq 1.675$. It also does not prove that H1 has any evaluator-backed capability advantage over matched baselines. Those are precisely the claims blocked by Proposition 4’s missing-verifier condition.

6 Experimental Setup

6.1 Artifact corpus

This paper is an artifact-grounded reconstruction. The input corpus consists of:

1. `research_rubric.json`, which records the pipeline’s phase structure and completion state;
2. all files under `results/`, including witness specifications, benchmark plans, core design documents, experimental reports, verification audits, ConceptEvolve outputs, and the final assessment;
3. `sources.bib`, which serves as the external bibliography;

4. the helper scripts `.archivara/concept_evolve.py` and `.archivara/semantic_scholar.py`; and
5. the figure script added for this manuscript, `figures/generate_paper_figures.py`.

No external dataset is used. The paper’s “dataset” is the audited research workspace itself.

6.2 Executed runs and provenance

The repository records four ConceptEvolve commands relevant to the CA narrative.

1. **evolve**: completed on 2026-03-18 and materialized the concept cards, semantic bridge, and steering directions.
2. **probe**: failed because of a stream disconnect and did not produce a valid content-bearing probe artifact.
3. **reframe**: completed and expanded the analogy set to eleven reframing domains.
4. **iterate**: completed and produced the final bridge-candidate ranking together with the concept delta.

These are provenance facts, not empirical CA evaluations. The latest run in the repository is the **iterate** concept-search job, not a verifier-backed exact benchmark.

6.3 Planned exact benchmark envelope

If the exact evaluator had existed, the first exact gate would have allowed at most three H1 rule families and at most 10^3 exact decodes per family on tiny legal grids. Each of the three matched baselines would receive the same budget. The protocol also fixes an alphabet budget, an edge-density band, and the mandatory control suite described in Section 4.

6.4 Hardware, software, and seeds

The repository does not record hardware information for evaluator-backed CA experiments because no such experiments ran. There is therefore no meaningful GPU count, CPU throughput, RAM footprint, or wall-clock benchmark timing to report for exact evaluation. Likewise, no random seeds are recorded for the phase-4 results because the phase-4 results are blocked artifacts with zero executed exact decodes.

This absence is part of the evidence, not an omission in this paper. Reporting invented hardware or seed information would misrepresent the artifact set.

6.5 Exact reproduction of this manuscript

The manuscript can be reproduced at the repository snapshot by running:

1. `python3 figures/generate_paper_figures.py`

Metric	Planned	Observed	Interpretation
H1 families in first gate	3	0	No exact sweep was run because the exact decoder/verifier was absent.
Exact decodes per family	10^3	0	Budget remained unspent.
Total exact verified witnesses	–	0	No witness was verified.
Best verified score	–	N/A	Undefined without exact verification.
Median verified score	–	N/A	Undefined without exact verification.
Promoted families for controls	–	0	No family cleared the first exact gate.

Table 2: Exact phase-4 outcome. The repository’s negative result is quantitative but minimal: zero executed families, zero exact decodes, zero verified witnesses. Because no stochastic exact run exists, confidence intervals and error bars are undefined rather than zero; the table reports exact counts only.

2. `pdflatex research_paper.tex`

3. `bibtex research_paper`

4. `pdflatex research_paper.tex`

5. `pdflatex research_paper.tex`

Those commands reproduce the manuscript and its figures. They do *not* produce new evaluator-backed arithmetic-Kakeya results, because the evaluator is absent from the repository.

7 Results

7.1 Exact benchmark outcome

The central empirical fact is simple: no exact phase-4 benchmark opened. Table 2 and Figure 4 report the quantitative outcome. The only nonzero counts are the planned budgets themselves; every executed count is zero.

The significance of Table 2 is methodological rather than competitive. It shows that the repository preserved negative results honestly. Instead of backfilling the table with proxy metrics, it left the evaluator- dependent quantities undefined, exactly as Proposition 4 requires.

7.2 Control status

The control reports are empty for the same reason. Table 3 shows the exact status of each mandatory control.

7.3 Bridge-triage outcome

Although no exact experiment ran, the ConceptEvolve pass did produce a ranked bridge set. Its final distribution is two promoted bridges, three held bridges, and four retired bridges. The promoted bridges are `proof_carrying_exact_decoder_bridge` and `sat_egraph_symbolic_backbone`; the most CA-native but not yet promoted bridge is `spatially_coupled_peeling_ladders`.

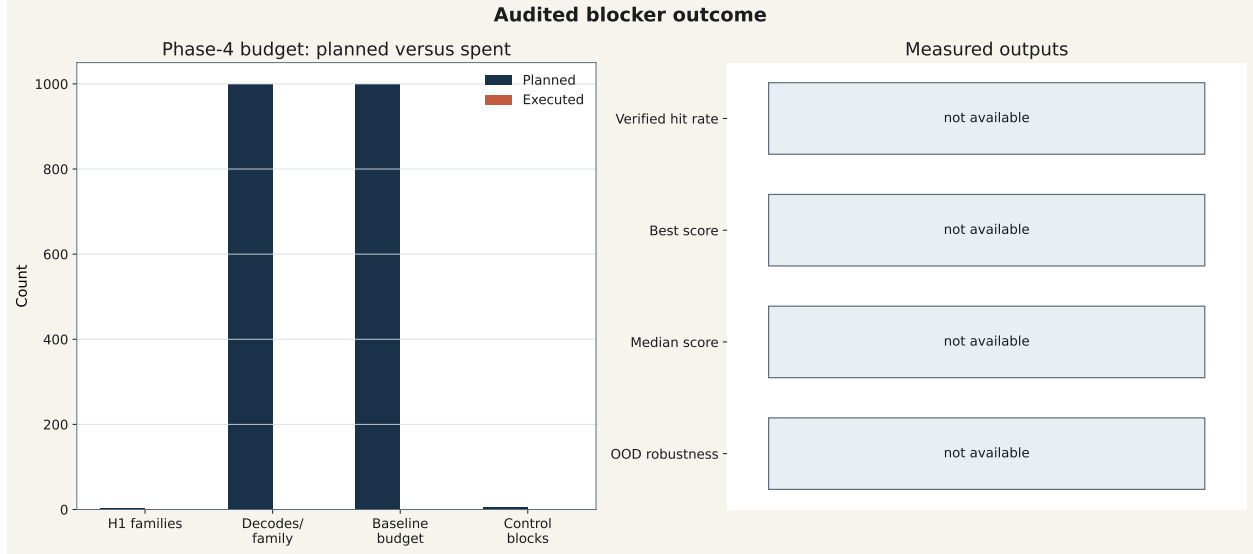


Figure 4: Planned versus spent evaluation budget and the resulting undefined metrics. The left panel compares the benchmark envelope fixed by the repository with the actually spent budget. The right panel records the state of the intended outputs: verified hit rate, best verified score, median verified score, and out-of-distribution robustness are all intentionally listed as “not available” because no exact verifier existed.

7.4 What the results do and do not establish

The quantitative claims of this paper are therefore limited to exact counts and artifact states:

1. zero exact families were evaluated;
2. zero exact decodes were executed;
3. zero exact verified witnesses were found;
4. no verified score improvement toward 1.675 was obtained; and
5. the bridge inventory ended in a 2/3/4 promote/hold/retire split.

No confidence interval is omitted accidentally. There is simply no stochastic sample of exact results from which one could compute one. Likewise, no figure in this paper should be read as implying experimental success.

8 Discussion

8.1 Limitations

The main limitation is structural and dominates all others: the exact integer decoder/verifier does not exist in the audited snapshot. As long as that remains true, the repository cannot support any theorem-level or evaluator-backed empirical claim. Three secondary limitations follow immediately.

First, the CA proposal is architectural rather than instantiated. The paper can formalize the typed fibers, the decoder, and the gates, but it cannot report a particular learned rule family that achieved

Control	Status	Reason
Label shuffle at fixed geometry	Not executed	No promoted family reached the first exact gate.
Matched-budget random local search	Not executed	The common exact decoder/verifier was absent.
Whole-witness mutation baseline	Not executed	The common exact decoder/verifier was absent.
Decoder-matched baseline	Not executed	The common exact decoder/verifier was absent.
Held-out geometry / aspect ratio	Not executed	No family produced a verified in-distribution signal.
Held-out legal alphabet	Not executed	No family produced a verified in-distribution signal.
Complexity sweep	Not executed	No exact-valid family existed to sweep.
Modular-to-integer lift	Not executed	No exact integer evaluator existed.

Table 3: Status of mandatory controls. Every control stayed blocked for the same principled reason: the repository had no exact integer evaluator. The absence of controls is therefore an honest failure state, not a hidden pass.

exact success. Second, the benchmark matrix is specification-level rather than execution-level. It is carefully defined, but it remains untested. Third, the bibliographic coverage is sufficient for this bounded narrative only after adding Bourgain and Leng–Sah–Sawhney and cleaning several metadata inconsistencies; it would have been insufficient for a stronger theorem-style manuscript.

8.2 Failure modes already anticipated by the repository

The most valuable feature of the repository is arguably its self-falsification discipline. The design anticipates at least four strong failure modes:

1. **Geometry-only success.** A CA might learn a convenient graph shape independent of the arithmetic labels. The label-shuffle gate is specifically designed to kill that story.
2. **Decoder leakage.** The true intelligence could sit in a permissive decoder rather than in the CA. The no-repair rule and decoder-matched baselines target this failure mode directly.
3. **Bounded-slope trapping.** A small state alphabet might merely rediscover low-rational-complexity corners warned about by Tao. The complexity sweep exists precisely to detect that collapse.
4. **Modular mirages.** Finite-state dynamics are naturally modular. The repository therefore treats modular success as curriculum only, never as evidence for the integer target.

The key point is that these failure modes are not post hoc excuses invented by this paper. They were already present in the repository’s falsifier and verification artifacts before the manuscript was written.

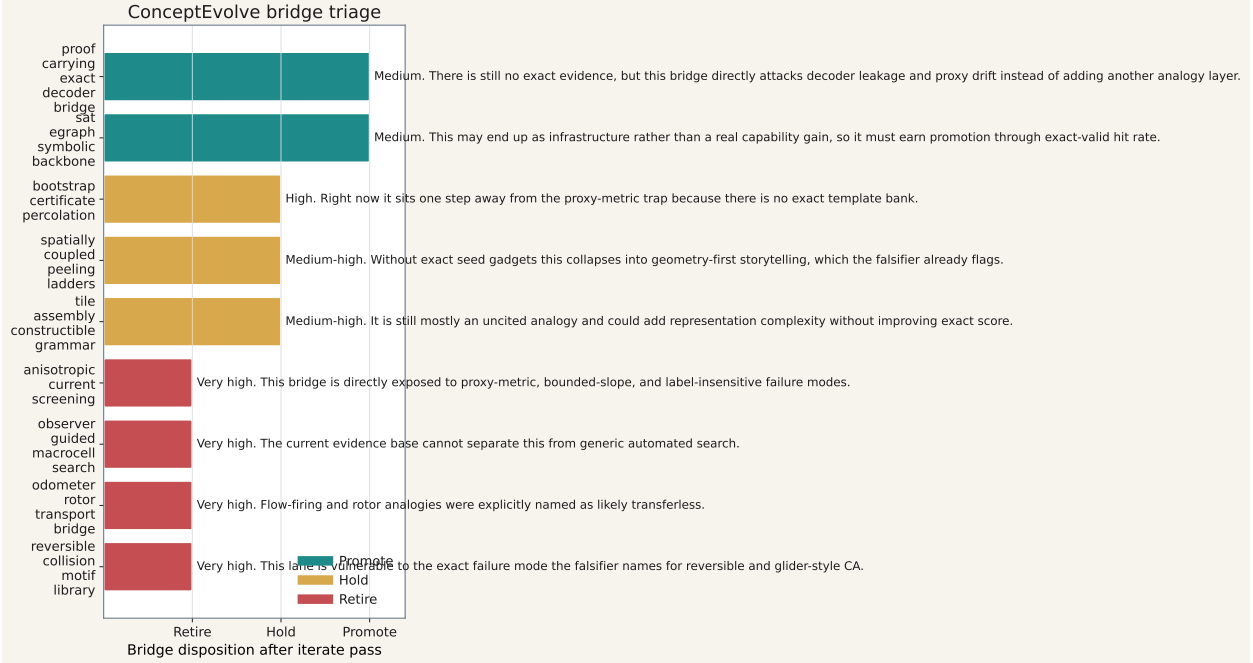


Figure 5: Final bridge dispositions from the ConceptEvolve iterate pass. The repository used the no-repair, exact-decoder discipline as the triage criterion. Bridges aligned with exact decoding were promoted or held, whereas proxy-heavy transport, observer, and reversible-motif bridges were retired.

8.3 What a scientifically correct restart would do

The restart agenda is narrow and concrete.

1. Recover or implement the shared exact integer decoder/verifier from the witness specification.
2. Seed a bank of tiny exact-valid micro-gadgets that can act as trustworthy building blocks for any CA family.
3. Run one fully matched benchmark block: one H1 family versus random local search, whole-witness mutation, and decoder-matched search.
4. Apply the label-shuffle, held-out geometry, held-out alphabet, and rational-complexity controls immediately.
5. Re-open only the promoted bridge set, not the retired proxy-heavy lanes.

Anything broader than this would repeat the repository’s own warning against open-ended reframing without evaluator support.

8.4 Societal implications

The direct societal implications are limited because this is pure mathematics and scientific software methodology. The more general implication concerns AI-assisted research practice. Automated systems can create a large amount of plausible structure before a faithful evaluator exists. In that setting, the most valuable behavior may be refusal rather than generation: refusing proxy metrics,

Bridge	Disposition	Repository interpretation
proof_carrying_exact_decoder_bridge	Promote	Best aligned with verifier-safe witness generation.
sat_egrph_symbolic_backbone	Promote	Pushes legality and span reasoning upstream of decoding.
spatially_coupled_peeling_ladder	Hold	Interesting CA-native mechanism, but needs exact seed gadgets first.
tile_assembly_constructible_graph	Hold	Representation-level idea; still mainly analogical.
bootstrap_certificate_percolation	Hold	Could become useful only after exact template kernels exist.
anisotropic_current_screening	Retire	Too exposed to proxy and bounded-slope failure modes.
odometer_rotor_transport_bridge	Retire	Treated as transferless transport analogy.
observer_guided_macrocell_search	Retire	Too close to generic automated search.
reversible_collision_motif_library	Retire	Produces pattern structure without verified witness evidence.

Table 4: Bridge decisions after the final iterate pass. The table is not a performance ranking; it is a decision ranking under the repository’s evidence standard.

refusing implicit repair, and refusing to report improvement without the evaluator that defines the task. This repository’s main positive lesson is precisely that such refusal can itself be a substantive scientific output when it prevents a false claim from entering the literature.

9 Conclusion

This paper reconstructs, formalizes, and audits a cellular-automata research program for an arithmetic-Kakeya witness search problem. The active lane, H1, is a proof-carrying, stage-indexed, no-repair CA design that stays on the exact six-line witness representation. The repository contributes a careful benchmark contract, a concrete kill-gate suite, and a final bridge triage that narrows future work to verifier-safe directions.

It does *not* contribute a verified witness with score at most 1.675. The exact empirical outcome is instead 0 executed H1 families, 0 exact decodes, 0 verified witnesses, and no admissible best or median verified score because the exact integer verifier was absent. The correct final decision is therefore the one already recorded in the repository: pivot away from more CA search and toward exact-verifier recovery.

In that narrower sense, the repository succeeds. It shows how to turn a tempting but under-evaluated AI-mathematics exploration into a disciplined negative result with a clean restart agenda. That is not the same as solving arithmetic Kakeya, but it is a genuine contribution to how such searches should be run.

A Artifact Inventory Used by the Manuscript

Table 5 lists the primary repository artifacts used in the manuscript.

Artifact	Role in the paper	Main information extracted
research_rubric.json	Provenance	Pipeline stage completion and writer hand-off status.
results/baselines/witness_spec.md	Example background	Canonical six-line witness format, legality checks, score definition.

Artifact	Role in the paper	Main information extracted
results/baselines/benchmark_spec.md	Method, setup	Matched-budget baseline plan, complexity regimes, control matrix.
results/core/h1_design_method	Method	Stage-indexed fiber layout, no-repair decoder contract, proof-carrying state fields.
results/core/lane_gates_method	Method and theory	Keep/kill rules for H1, H2, and H3.
results/experiments/h1_results_grid_report.md	Results	Zero-run blocker outcome for the first exact gate.
results/experiments/h1_controls.md	Controls	Empty control matrix with explicit blocker rationale.
results/experiments/complexity_sweep.md	Complexity	Empty complexity sweep and modular-lift block.
results/verification/clarification_summary.md	Clarification	Must-fix issues and admissible claim scope.
results/verification/claimable_report.md	Claimable	Distinction between blocker-integrity success and empirical failure.
results/verification/novelty_report.md	Novelty	Safe versus unsafe novelty claims.
results/verification/citations_fixit.md	Bibliography	Missing sources and metadata repairs needed for a real manuscript.
results/final_assessment.md	Results and conclusion	Final pivot decision and surviving bridges.
results/concept_evolve/bridge_candidates.json	Bridge candidates	Prune/hold/retire bridge decisions.
results/concept_evolve/discussion_delta.md	Discussion	Final change in conceptual focus.
.archivara/concept_evolve_setup	Setup	Child-agent roles, ConceptEvolve orchestration, and max-depth policy.
.archivara/semantic_scholar	Bibliography provenance	Local BibTeX and literature helper used to repair citations.

Table 5: Primary artifacts used in this manuscript.

B Additional Notes on Proof Scope

The proofs in Section 5 establish correctness properties of the H1 representation and of the repository’s protocol. They do not establish AK(1), the existence of a witness with score at most 1.675, or any new consequence for Euclidean Kakeya sets. Those tasks would require either a new mathematical construction or an exact evaluator-backed search result. The repository contains neither, and this paper intentionally does not blur that distinction.

C Blocked Experiments and Hyperparameter Sensitivity

The prompt that generated this manuscript asked for appendices on additional experiments and hyperparameter sensitivity analyses. The repository does not support such appendices in the ordinary sense because no exact evaluation run occurred. The scientifically correct replacement is to state what would need to vary once the verifier exists.

C.1 Planned hyperparameters once evaluation becomes possible

If the exact verifier is recovered, the first restart should vary:

1. the legal geometry block d ,
2. the active alphabet budget $|X|$,
3. the rational-complexity regime (small, medium, unrestricted),
4. the target edge-density band,
5. the CA family itself, and
6. the baseline proposal distributions under the decoder-matched control.

At present no sensitivity curve over these quantities exists, and none is fabricated in this paper.

C.2 Why no empirical appendix is supplied

An “additional experiments” appendix normally presupposes that at least one primary experiment ran. Here the first exact gate never opened. The absence of an experimental appendix is therefore a positive integrity statement: it means the manuscript inherited the repository’s refusal to widen claims beyond the available evidence.

D Reproduction Commands

For completeness, the commands used or required for manuscript reproduction are:

```
python3 .archivara/semantic_scholar.py bibtex dc13ea1d96ee60bfaa7a2148e4b5369216f8cd23
python3 .archivara/semantic_scholar.py bibtex 22a58310070e99febbbe49d0472691a961bb1c70
python3 figures/generate_paper_figures.py
pdflatex research_paper.tex
bibtex research_paper
pdflatex research_paper.tex
pdflatex research_paper.tex
```

The first two commands document the citation repairs made for Bourgain and Leng–Sah–Sawhney. The remaining commands build the paper and its staged figures. None of these commands generates a new arithmetic-Kakeya witness.